

Brief Introduction to Natural Language Processing

VIP IP Crowdsourcing Spring 2022

Carl Cox, Tiffany Huang

Overview

- ▶ What is Natural Language Processing (NLP)?
- ▶ What are some applications in NLP?
- ▶ Some NLP work at GTRI
 - ▶ Automatic Ontology Learning
 - ▶ ASIST
 - ▶ Analysis of Human Behavior for Domestic Violence Prevention/Mitigation (DVPM)
- ▶ Software of interest
 - ▶ Python library/codebase
 - ▶ Non-Python library/codebase
- ▶ Python Jupyter notebook tutorials

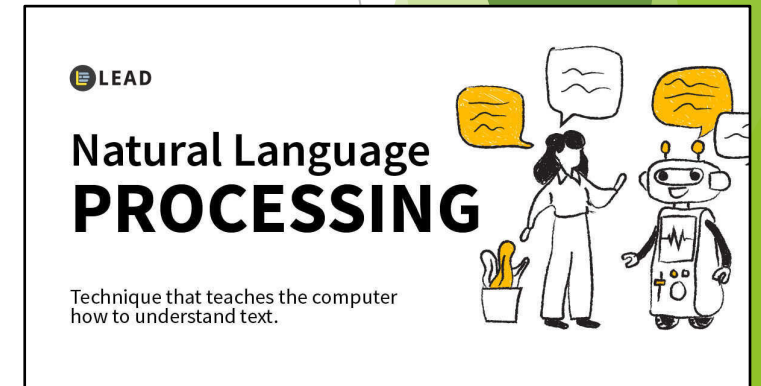
What is Natural Language Processing (NLP)?

- Natural Language Processing (NLP) is part of Artificial Intelligence (AI) that allows machines the ability to understand and analyze texts from human languages (more automated process and less human in the loop).

A review of Restaurant ABC:

The new menu **Chicken Spaghetti** is of **high quality, tasty** and in **decent price**. **Will** definitely **come back** again!

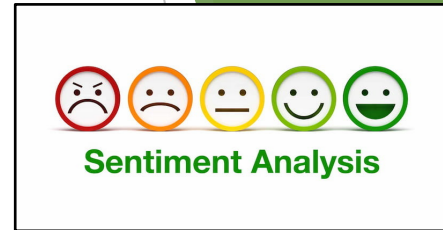
After a NLP model -> It is likely a positive review.



<https://thelead.io/data-science/what-is-natural-language-processing>

What are some applications in NLP?

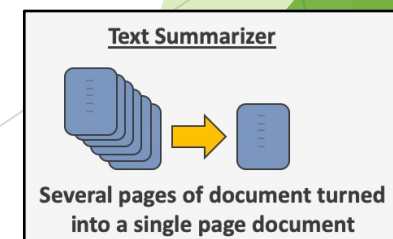
- ▶ Sentiment Analysis
 - ▶ Be able to classify texts to either positive or negative sentiment (binary classification)
- ▶ Topic Modeling
 - ▶ Determine a set of topics from a corpus (clustering)
- ▶ Email spam filters
 - ▶ Filter out or classify spam emails
- ▶ Text summarization
 - ▶ Summarize or simplify long document into smaller amount of text
- ▶ Identify misinformation or machine generated text from human



<https://blog.reputationx.com/reputation-sentiment>

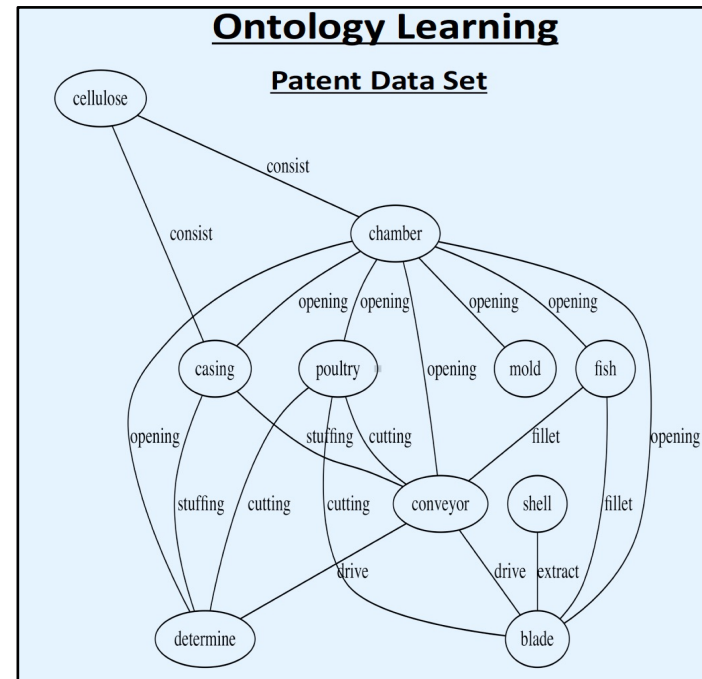
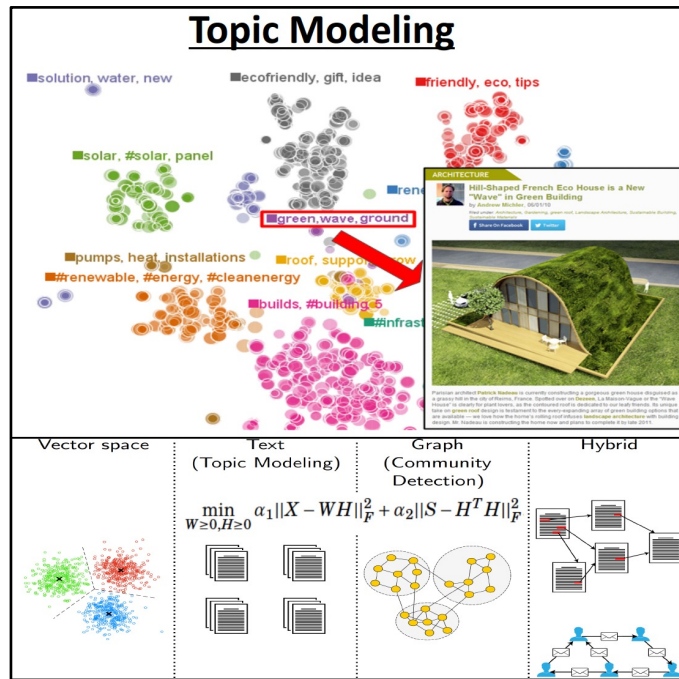


<https://dataaspirant.com/build-email-spam-classification-model-spacy-python/>



Some NLP work at GTRI

- Automatic Ontology Learning
 - From labor-intensive ontology construction to automated ontology learning



Some NLP work at GTRI

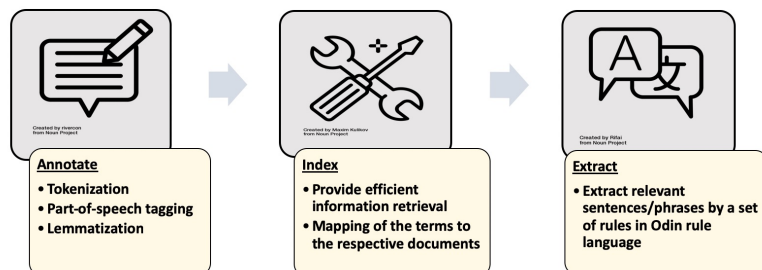
► ASIST

- Given a corpus of research publications, extract directional hypothesis sentences

truth	casual-rule-1	casual-rule-2	effect-noun-adjective-long	effect-noun-adjective	effect-comparative-long	effect-comparative	max-similarity	odinson
The amount of inspected information increases with an increasing number of reasons that have to be considered.	0.247706422	0.211009174	0.706422018		0.247706422	0.183486239	0.706422018	Decision Time Hypotheses H2a (PH) : Decision time increases with an increasing number of reasons that have to be considered .
Later in a team's life cycle, dis-agreements are less likely to reveal unknown, major underlying differences in assumptions and mental models.	0.314285714	0.242857143	0.314285714	0.992857143	0.135714286	0.321428571	0.992857143	Later in a team ' s life cycle , dis - agreements are less likely to reveal unknown , major underlying differences in assumptions and mental models .

Odinson

An open-source ruled-based framework for information extraction written in Scala. Three main steps:



```
rules:

- name: effect-noun-adjective
  type: basic
  pattern: |
    (?<subject>[tag=/N.*/]) [tag="MD"]*?
    (?<trigger>[tag=/VB.*/])
    [tag="DT"]*? (?<direction>[word=/positive|negative|higher|lower|high|low|better|worse/])
    (?<event>[tag=/N.*/])
    [tag="IN"]*? (?<predicate>[tag=/N.*/])

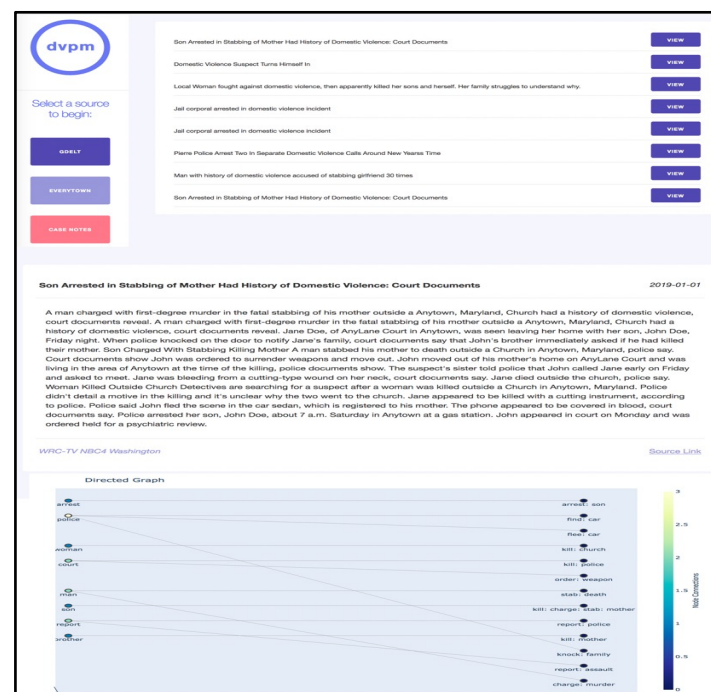
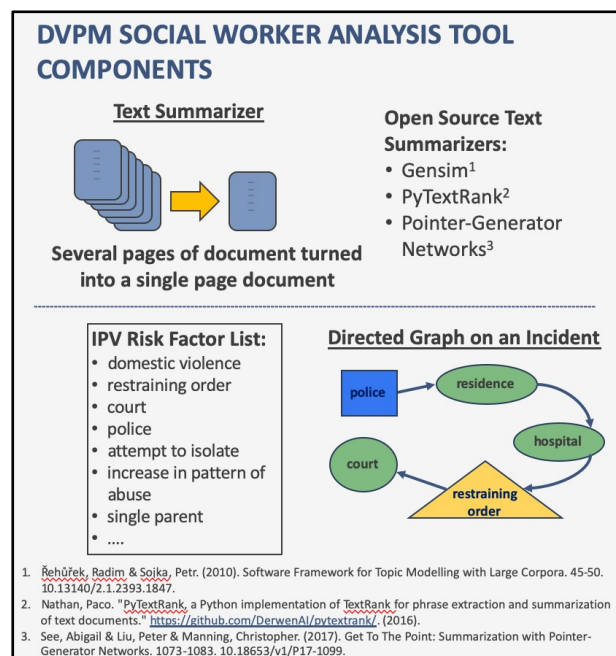
- name: effect-comparative
  type: basic
  pattern: |
    (?<subject>[tag=/N.*/]) [tag="MD"]*?
    (?<trigger>[tag=/VB.*/])
    (?<direction>[tag=/RBR|JJR/])
    [tag="JJ"]*? (?<predicate>[tag=/N.*/]) [tag="IN"]*?

- name: effect-adverb
  type: basic
  pattern: |
    (?<subject>[tag=/N.*/]) [tag="MD"]*?
    (?<trigger>[tag=/VB.*/])
    (?<direction>[word=/positive|negative|higher|lower|high|low|better|worse/])
    (?<predicate>[tag=/N.*/])

- name: effect-ex-adjective
  type: basic
  pattern: |
    (?<subject>[tag="EX"]) [tag="MD"]*?
    (?<trigger>[tag=/VB.*/])
    [tag="DT"]*? (?<direction>[word=/positive|negative|higher|lower|high|low|better|worse/])
    (?<event>[tag=/N.*/])
    [tag="IN"]*? (?<predicate>[tag=/N.*/])
```

Some NLP work at GTRI

- ▶ Analysis of Human Behavior for Domestic Violence Prevention/Mitigation (DVPM)
 - ▶ Support domestic violence professionals gaining intelligence from unstructured text in an automated fashion



Software of interest

► Python library/codebase

- **spaCy** - [spaCy · Industrial-strength Natural Language Processing in Python](<https://spacy.io/>)
- **PyTextRank** (text summarizer) - [GitHub - DerwenAI/pytextrank: Python implementation of TextRank for phrase extraction and summarization of text documents](<https://github.com/DerwenAI/pytextrank/>)
- **Gensim** (Topic Modeling) - [Gensim: Topic modelling for humans](<https://radimrehurek.com/gensim/>)
- **re** (Python build-in regular expression library) - [re — Regular expression operations — Python 3.10.1 documentation](<https://docs.python.org/3/library/re.html>)
- **Spark NLP** (NLP library similar to spaCy - [John Snow Labs - Spark NLP](<https://nlp.johnsnowlabs.com/>)
- **skweak** (document labeling model) - <https://github.com/NorskRegnesentral/skweak>
- **Yake** (Yet Another Keyword Extractor) - [GitHub - LIAAD/yake: Single-document unsupervised keyword extraction](<https://github.com/LIAAD/yake>)
- **trankit** - [GitHub - nlp-uoregon/trankit: Trankit is a Light-Weight Transformer-based Python Toolkit for Multilingual Natural Language Processing](<https://github.com/nlp-uoregon/trankit>) and its demo site: [Trankit's Demo Website](<http://nlp.uoregon.edu/trankit>)
- **Neural Semi-Markov CRF (Conditional Random Field) for Monolingual Word Alignment** (GT authors) (Monolingual word alignment is important for studying fine-grained editing operations (i.e., deletion, addition, and substitution) in text-to-text generation tasks, such as paraphrase generation, text simplification, neutralizing biased language, etc.)
 - [2106.02569 Neural semi-Markov CRF for Monolingual Word Alignment](<https://arxiv.org/abs/2106.02569>) (paper)
 - [GitHub - chaojiang06/neural-Jacana: This is the code for neural-Jacana aligner, and the data for MultiMWA dataset.](<https://github.com/chaojiang06/neural-Jacana>) (code)
- **MadDog: Web-based System for Acronym Identification and Disambiguation**
 - <https://github.com/amirveyseh/MadDog> (code)
 - <http://iq.cs.uoregon.edu:5000/> (demo)

Software of interest

► Non-Python library/codebase

- Odinson (rule-based information extraction framework) - [Overview - odinson](<http://gh.lum.ai/odinson/>)
- Apache Solr (Indexing and tokenization) - [About This Guide | Apache Solr Reference Guide 6.6](https://solr.apache.org/guide/6_6/about-this-guide.html)
- SmallK (Topic Modeling based on NMF by GT and GTRI) - [SmallK — SmallK 1.6.2 documentation](<https://smallk.github.io/>)
- LEAN (Lucene Tools for Text Analytics) - [GitHub - smallk/lean: Lucene Tools for Text Analytics](<https://github.com/smallk/lean>), generate term-frequency matrices from text documents
- GLTR: Giant Language Model Test Room (detecting machine-generated text from text written by real humans) - [GitHub - HendrikStrobelt/detecting-fake-text: Giant Language Model Test Room](<https://github.com/HendrikStrobelt/detecting-fake-text>)

Python Jupyter notebook tutorials

- ▶ spaCy
 - ▶ `spacy_nlp_linguistic.ipynb`
 - ▶ `spacy_nlp_matching.ipynb`
 - ▶ `spacy_nlp_pipeline.ipynb`
- ▶ skweak
 - ▶ `skweak_nlp_example.ipynb`

Potential approaches on generating context-based hashtags (Pure NLP approach)

- ▶ Utilize word embedding or word2vec technique in NLP to classify a given text to either of the binary classes: animal vs. sport for example
- ▶ For instance, the large model “en_core_web_lg” from spaCy package contains 685k unique vectors that represent 685k terms in English, where two terms are said to be similar if their differences in distance are small. Here is a code snippet from the spaCy documentation <https://spacy.io/usage/linguistic-features#vectors-similarity>:

0.78 is the overall similarity between the two text.

```
Editable Code spaCy v3.0 · Python 3 · via Binder

import spacy

nlp = spacy.load("en_core_web_md") # make sure to use larger package!
doc1 = nlp("I like salty fries and hamburgers.")
doc2 = nlp("Fast food tastes very good.")

# Similarity of two documents
print(doc1, "<->", doc2, doc1.similarity(doc2))
# Similarity of tokens and spans
french_fries = doc1[2:4]
burgers = doc1[5]
print(french_fries, "<->", burgers, french_fries.similarity(burgers))

RUN

I like salty fries and hamburgers. <-> Fast food tastes very good.
0.7799485853415737
salty fries <-> hamburgers 0.7304624
```

Therefore, for a given text mentioning Atlanta Hawks game, if compared to another text about the game, it is assumed that the similarity would be higher than comparing it to a text that talks about hawks as animals.

If the existing model from spaCy is not sufficient, a customized model may need to be trained:

<https://spacy.io/usage/linguistic-features#adding-vectors>

Potential approaches on generating context-based hashtags (ML approach)

- ▶ Build a classification model such as Support Vector Machine (SVM) or Logistic Regression on classifying a given text to either of the binary classes: animal vs. sport for example
 - ▶ Step 0: Obtain a text corpus related to the topic of interest to be the training dataset. For example, short texts from Reddit that mention the topic of Atlanta Hawks game (1st class), as well as texts that mention the hawks as animal (2nd class). Assign a label (0 and 1) to each text according to their class (the truth labels)
 - ▶ Step 1: Determine a set of features that would help the model to classify text to either of the two classes^{1,2,3,4} such as a TF-IDF or term-frequency (TF) features for example
 - ▶ Apply NLP to preprocess or prune the texts so only relevant terms are used for the features. The model's computation is more efficient and accurate if the step is done well. Potential techniques are:
 - ▶ Stemming
 - ▶ Filtering out stopwords, punctuations, etc., if applicable
 - ▶ Pattern matching
 - ▶ Any other techniques that help to build the features

- ▶ Build a classification model such as Support Vector Machine (SVM) or Logistic Regression on classifying a given text to either of the binary classes: animal vs. sport for example
 - ▶ Step 2: Create an input matrix for the model based on the features from step 1. Note that the matrix is a numeric-value representation of the features. If number of documents is large enough, the matrix could be split into a training matrix and a testing/validation matrix
 - ▶ Step 3: Train a classification model of choice using the training matrix created from step 2 and their respective truth labels. Test or validate the model using the testing/validation matrix if available
 - ▶ If the accuracy from the testing is not satisfying:
 - ▶ The features from step 1 may need to be adjusted by adding new or removing existing feature(s)
 - ▶ Try adjusting the model's parameters
 - ▶ Try another classification model
 - ▶ Step 4: For a new text, classify the text to either of the two classes by the model build from step 3
 - ▶ Represent the text in terms of the numeric-value presentation of the features used in training
 - ▶ Use the resulting coefficients/parameters from the built model to classify the text
 - ▶ If the model is trained using 0 and 1 as truth labels, we could then classify the new text to 1st class if the result is between $[0, 0.5]$ and to 2nd class if the result is between $(0.5, 1]$
 - ▶ Note that here we assume the first class has 0 as truth label, whereas the second class has 1 as truth label

- ▶ There are some research publications that may help to build the features:
 1. “A survey on extremism analysis using natural language processing: definitions, literature review, trends and challenges” by Torregrosa, Bello-Orgaz, Martínez-Cámara, Del Ser and Camacho:
<https://link.springer.com/content/pdf/10.1007/s12652-021-03658-z.pdf> (Section 5 on features)
 2. “Automatic Detection of Fake News” by P´erez-Rosas, Kleinberg, Lefevre and Mihalcea: <https://web.eecs.umich.edu/~mihalcea/papers/perezrosas.coling18.pdf> (Section 4 on features)
 3. “Box of Lies R©: Multimodal Deception Detection in Dialogues” by Soldner, P´erez-Rosas and Mihalcea:
<https://lit.eecs.umich.edu/files/multimodalDialogDeception.pdf> (Section 4 on features)
 4. “Experiments in Open Domain Deception Detection” by P´erez-Rosas and Mihalcea:
<https://web.eecs.umich.edu/~mihalcea/papers/perezrosas.emnlp15.pdf> (Section 4 on features)